

The Missing Backend for the Future of Knowledge Work

The problem

AI-native has become table-stakes language. In practice, the term stretches from *"we paste meeting notes into ChatGPT"* to *"we have full trust in our agents acting on our systems of record."* The reason is simple. Context is scattered across the organisation's tools, locked in silos. The models are absurdly powerful, but only if they have all the details for the intended use. Today, they almost never do.

The autonomous company scale is a useful way to read where an organisation actually sits.

Level	What AI sees	What AI does	Hard test
L0	Nothing	Nothing	Can AI complete any process? <i>No.</i>
L1	What one person feeds it	Drafts, summarizes	If your best AI user left, would the workflow survive?
L2	Shared org context	Reads everything, can't act	Can a new joiner reach full deal context in 30 minutes?
L3	All systems, cross-linked	Acts on CRM, drafts docs, routes tasks	Can an agent summarize the week and take the next actions?
L4	Relationships between events	Runs processes, improves itself	Did a workflow improve without a human tuning it?
L5	Generative, asks its own questions	Delegated authority	Did the system notice something no one asked about?

Most stall somewhere between L1 and L3, and they stall for the same reason: a missing shared context layer underneath.

The shape of the solution

A central context maintenance engine, lab-agnostic, delivered as a managed service. It sits underneath every tool, every channel, every agent. The closest

mental model is autonomous git. We need the same primitives, made native to knowledge work, running in the background.

It needs to deliver two things:

1. Autonomous context management that produces a tangible boost in work effectiveness and token efficiency, for both agents and human workers. With the approach working for small companies as well as large enterprises.
2. A true feeling of control and increased trust in AI applications and their outputs.

Single connectors and MCPs are not close to doing the full job.

The user experience must-haves

A context engine is only worth as much as the trust people place in it. That trust is set by both the graph and the interface. Even the best ontology fails if the human layer feels like work. People stop curating, context drifts, and the system decays.

Enterprise adoption is won or lost on friction. This layer is often under-built because it looks like polish, but it determines whether the system stays accurate.

The engine's mental model is git: agents propose changes, humans review, changes merge. That is the right engine but the wrong interface. Branches, diffs, and merges are developer concepts. Ops and GTM leaders will not review pull requests. The experience layer has one job: keep git-grade rigour underneath, while the surface feels like Notion. Every change must be branchable, reviewable, revertible, and auditable.

Suggest, don't make people author. Blank-box editing is high friction. The system should draft, and humans should approve, tweak, or dismiss in one gesture. Either in natural language, or with rich text editing. A suggestion is the consumer version of a pull request.

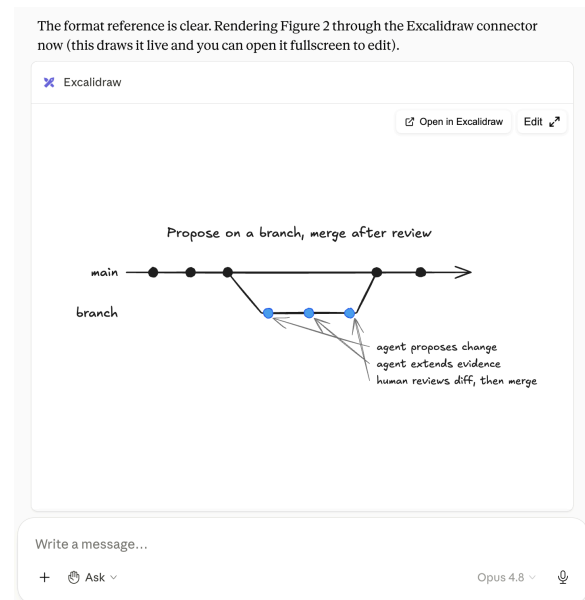
The unit of control is an artifact, not a field. The system should propose durable, named pieces of knowledge, like preferences and "how we do X". Each can be accepted or rejected, edited, and toggled on or off later.

This lives on two surfaces:

(1) Ambient, in the flow of work.

Context should be curated where work happens. Thus the UX should be a control panel that opens where work already happens. Excalidraw has a very nice dynamic connector, basically a “dynamic iframe” within the normal chat interface. I believe this would be great here as well. Nobody wants to switch apps / interface.

(2) A traditional web-app. The second surface is a Notion-like home base where the organisation’s context lives as readable pages and databases. Leadership can see what the system knows, where it came from, and what changed.



Functional must haves

1. **Autonomous context capture and curated retrieval.** Continuously unifies context across all apps and channels, traditional tools and AI productivity applications alike.
2. **Enterprise-grade compliance and access permissioning.** Conformity with the EU AI Act, GDPR, SOC2 & Co.
3. **Effective reduction of vendor lock-in.** Provider-agnostic by design. Customers should be able to swap the model layer underneath without problems. This is a key selling point.
4. **Immutable audit logs and an autonomous improvement loop.** Every action traceable. The system improves on its own as it runs.

Existing Companies / Projects

<https://github.com/volcengine/OpenViking>

<https://modernrelay.com/>